Batch Processing

bounded data, rerunnable results

Watch the 4-min video

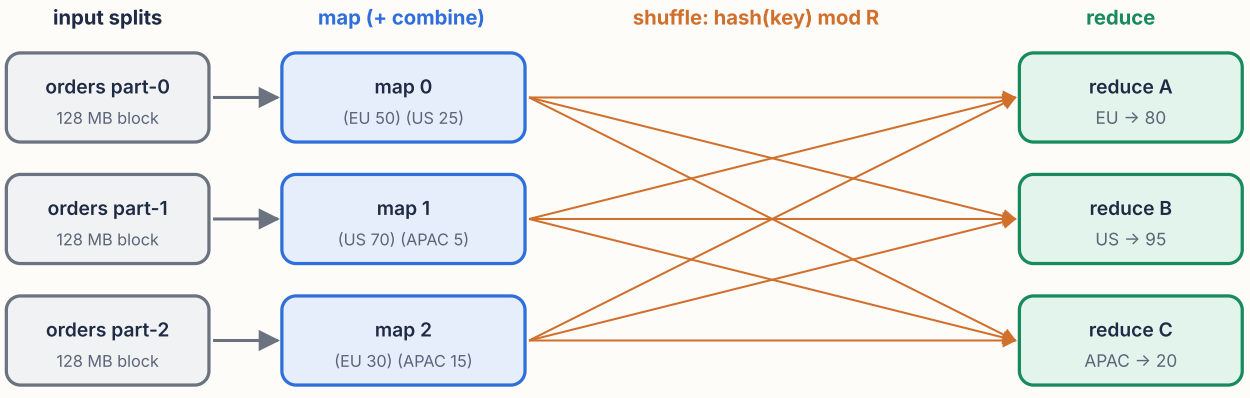
In one line: a batch job runs over a **bounded**, fixed input (such as yesterday's partition) and produces a complete result; because the input doesn't change, the job can be **rerun** to get the same answer, which makes batch the cheapest and easiest-to-correct way to compute results that may be hours old.

1 Yesterday's revenue, complete and correct

At 8 a.m. the sales team opens a dashboard of yesterday's revenue by region, computed from 300 million order rows. Nobody needs it second by second, but it must be complete (every order), correct (refunds, currency) and cheap. That is batch processing: wait until a period of data is closed, then process all of it at once on a cluster that can be shut down afterwards.

	Batch	Stream (lesson 32)
Input	bounded: a file set, a day's partition	unbounded: events forever
Latency	minutes to hours	milliseconds to seconds
Correction	fix code, rerun the partition	replay the log with a new job version
Hard parts	shuffle cost, skew, scheduling	state, event time, late data
Typical uses	reports, billing, ML training, backfills	fraud, alerts, live metrics

2 MapReduce: the model behind it



the shuffle moves every map output across the network, sorted and grouped by key: usually the slowest step

Figure 1. Revenue by region. Each map emits (region, amount) and pre-sums locally (combiner); the shuffle sends each key to one reducer, which sums it.

From Google's 2004 paper (Dean & Ghemawat): **map** reads records from input splits and emits key-value pairs; the framework **shuffles** them so all values for one key reach the same reducer (partitioned by `hash(key) mod R`, sorted by key); **reduce** combines each key's values. Failed tasks are simply re-executed on another machine, which works because tasks are deterministic and write output only on success.

- **Combiner:** a mini-reduce on the map side (sum per region before sending), which can cut shuffle volume by orders of magnitude.
- **Data skew:** if one key (a huge region, a null ID) holds most of the data, one reducer runs for hours while others idle. Fixes: salt the hot key into N sub-keys and merge, or handle it separately.
- **Data locality:** schedule map tasks where the data blocks live (HDFS) to avoid reading them over the network.

3 Spark: the usual engine today

Spark keeps intermediate data in memory where possible and plans the whole job as a **DAG of stages**. Transformations are lazy; when an action runs, the optimiser (Catalyst, for DataFrames and SQL) builds a plan. **Narrow** transformations (filter, map) stay in one stage; **wide** ones (groupBy, join) need a shuffle and start a new stage. Classic MapReduce wrote to disk between every step; a Spark pipeline of five steps may shuffle only once or twice.

- **Columnar files:** Parquet or ORC store each column separately with min/max statistics, so a query reads only the columns it needs and skips row groups that can't match (predicate pushdown).
- **Separate storage and compute:** read from S3/GCS/ADLS, run on a cluster sized for this job (often spot instances, since failed tasks just retry), then release it.
- **Tuning knobs:** number of shuffle partitions (`spark.sql.shuffle.partitions`, default 200), broadcast joins for small tables, and Adaptive Query Execution, which can coalesce partitions and split skewed ones at run time.

4 Partitioned and idempotent

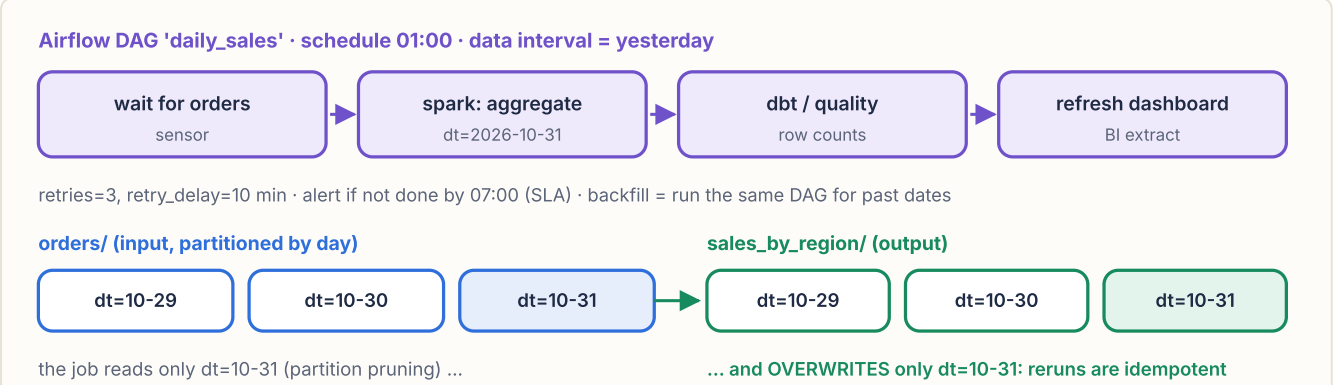


Figure 2. The job's input and output are both partitioned by day. Each run reads one input partition and replaces one output partition.

Store data in partitions such as `orders/dt=2026-10-31/`. A job then reads only what it needs (**partition pruning**) and writes to exactly one output partition by **overwriting** it, never appending. Rerunning yesterday after a bug fix replaces yesterday's results; appending would double them. In Spark, dynamic partition overwrite (`partitionOverwriteMode=dynamic`) or a table format with atomic commits (Delta Lake, Iceberg; lesson 34) makes the swap all-or-nothing, so readers never see a half-written partition. Also keep jobs deterministic: no `now()` inside the logic, use the run's logical date instead.

5 Orchestration

Jobs depend on each other: extract, then aggregate, then check, then refresh the dashboard. An orchestrator such as **Airflow** defines these as a DAG of tasks with a schedule and gives you: dependency ordering, **retries** with delays, **sensors** that wait for input to land, **backfills** (run the DAG for a range of

past dates), SLAs and alerts when something is late, and a history of every run. In Airflow, a daily run covers a **data interval** and starts after that interval ends, so the 01:00 run on Nov 1 processes Oct 31.

WORKED EXAMPLE: NIGHTLY SALES BY REGION

At 01:00 Airflow starts DAG `daily_sales` for interval Oct 31. A sensor confirms `orders/dt=2026-10-31` is complete. A Spark job reads that partition (300M rows, only the columns region, amount, currency, status), converts currency, drops cancelled orders, groups by region and sums: one shuffle of a few hundred KB after the combiner. It overwrites `sales_by_region/dt=2026-10-31`; a quality task checks row counts and totals against yesterday; the dashboard refreshes by 07:00. On Nov 6 a currency-conversion bug is found: the team fixes the code and runs a backfill for Oct 30 – Nov 5. Seven partitions are overwritten and every number is corrected, with no duplicates.

TRADE-OFFS

- Latency for simplicity: results are hours old, but cheap, complete and easy to fix.
- Bigger clusters finish faster but cost more; the shuffle, not CPU, usually limits speed.
- Small, frequent partitions give fresher results but create many small files and more scheduling overhead.
- Waiting longer before running catches more late data but delays the report.

COMMON MISTAKES

- Appending output, so every rerun doubles the numbers.
- Using wall-clock time (`now()`) inside the job, so a backfill computes the wrong day.
- Ignoring skew: one hot key makes a 10-minute job take hours.
- Starting the job on a timer without checking that the input is complete.

6 Key terms

Bounded data: a fixed input with a beginning and end

Combiner: map-side pre-aggregation that shrinks the shuffle

Idempotent rerun: running again replaces, never duplicates, output

Shuffle: moving map output across machines, grouped by key

Partition pruning: reading only the partitions a query needs

Backfill: running a pipeline for past dates

7 Interview questions

Q1. When is batch processing preferable to streaming? BASIC

When hours-old results are fine, such as daily reports, billing and model training, and you value simplicity, low cost, complete data and easy reprocessing.

Q2. What makes a batch job safe to rerun? BASIC

Partitioned inputs and outputs, with each run overwriting its own partition deterministically, so reruns are idempotent.

Q3. What are map, shuffle and reduce? BASIC

Map emits key-value pairs per record; shuffle groups all values for each key onto one machine; reduce combines each group.

Q4. A Spark job's last task runs for two hours while the others finish in minutes. Why? SCENARIO

Data skew: one key holds most rows. Salt the key, broadcast the small side of a join, or enable adaptive skew handling.

Q5. A bug corrupted a month of daily summaries. How do you fix it? SCENARIO

Fix the code, then backfill those 30 dates; each run overwrites its partition, so results are replaced without duplicates.

Q6. Why is Spark faster than classic MapReduce for multi-step jobs? TRICKY

It plans the whole DAG, pipelines narrow steps in memory and only materialises at shuffles, while MapReduce wrote to disk between every job.

Q7. Your nightly job sometimes misses orders that arrive late. What do you do? TRICKY

Wait for a completeness signal before running, and periodically re-run the last few days so late records land in the right partition.

Next: SD-Q-007 · Stream Processing backendroadmap.com